

SYM-2P: A Small Message Language for Teams of Agents

A reader's guide to the shipped protocol, its conventions, and its limits

Profile	SYM-2P/1.0
Wire version	v = integer 2
Carrier	one canonical JSON packet per line
Public home	the protean-sym2p repository

Revision: September 17, 2026. Audience: platform engineers and agent engineers who need to read a SYM-2P stream, validate one, or decide whether to adopt the protocol. Companion documents, all in the repository: the normative SPEC.md, the usage skill at skills/sym2p/SKILL.md, and the evidence notes under AUDIT/protean-sym2p/.

Three lines, if that is all you read. SYM-2P is a small message language for teams of agents that share work through files. It carries one canonical JSON packet per line inside durable text artifacts. Each packet states the smallest state change that lets the receiving agent act correctly.

Design objective. Communicate the smallest sufficient state change that lets the receiving agent act correctly. Every rule below is an answer to that one sentence.

If you read an earlier draft of this report, Appendix B maps the earlier sections onto the sections here. The report itself is written as a guide, not as a change log.

Contents

1. In one minute	3
2. Why it exists	4
3. The shape of a team	4
4. Rules that keep compression safe	4
5. The packet	5
6. Vocabulary	6
7. Durable state and versions	7
8. Deltas, retries, and failure	8
9. Routing and the stop rule	9
10. Uncertainty and dissent	10
11. The human boundary and authorization	11
12. Errors and the fallback ladder	12
13. Prompt contracts	13
14. Worked flow	14
15. Implementation status	15
16. How to use the repository	17
17. Limits to build within	18
Appendix A. Quick reference	19
Appendix B. What moved from the earlier drafts	19

1. In one minute

SYM-2P is a versioned packet language, a set of file conventions, and a strict validator. One line of JSON per packet. One act per packet. One recipient per packet. The content lives in durable objects; the packet carries the smallest change and a reference to the rest.

The positioning, in three parts	
What it is	A versioned packet language plus file conventions plus a strict validator. One canonical JSON object per line, one act per packet, one recipient per packet. Objects hold content; packets hold references and the minimum change.
When to use it	When agents coordinate through durable files and every consequential step must later be reconstructed from the record. When bandwidth matters but audit matters more. When independent checks must run before irreversible actions. When a receiver must validate a stream against receiver state, or prove canonical form, and get typed diagnostics with exit codes 0, 1, and 2.
When to skip it	When one agent can do the job. When the team needs live negotiation with no record. When the platform already provides authenticated messaging with its own schema and the cost of a second layer exceeds its value. When the run depends on any open item O-1 to O-9. When only record schemas and gates are wanted, which ship in a separate operations ingredient.

Where this stands today

Phase 2 is adopted and implemented. Schema validation, object storage conventions, versioning, references, deltas, idempotency, error packets, and the fallback ladder ship in the repository and are exercised by its gates.

>

Phase 3 routing optimization is held, not implemented. Its novelty, expected-value, and cost predicates are undefined and unverified. This report does not market the hold away.

>

Nine open items stay open. O-1 to O-9 are carried open in the specification. None is closed by this release, and none is presented as decided.

What this report does not claim

No live blackboard service ships with this release. No live multi-agent deployment was used to verify it. Transport signing is not implemented. Confidence scores are not calibrated probabilities. Phase 3 routing is not implemented. There is no full platform authorization store. Where the repository records an assumption rather than a fact, this report says so in the same words.

2. Why it exists

Agent teams fail in familiar ways. They repeat shared history in every message. They overwrite a shared plan without a version. They lose the losing view when two agents disagree. They act on approval that was vague when it was given. They guess when a message arrives broken.

Each failure has a mechanism here.

References replace repetition. Explicit base versions replace silent overwrites. Dispute objects preserve dissent. Approval travels as a scoped reference with an expiry. Broken input gets a typed error, never a guess.

The bet behind all five is small and testable. If the unit of communication is the smallest state change that lets the receiver act correctly, then the record stays reconstructable, the messages stay short, and the failure modes stay visible. If that bet is wrong, it will be wrong in a way the receipts can show.

3. The shape of a team

Four parts take part.

- A **relay** faces the human in English, turns English into scoped packets, and attaches approval.
- A **coordinator** owns the task plan, sets acceptance criteria, routes work, and records decisions.
- **Specialists** do bounded work, publish evidence, and return short packets.
- A **tree of files** holds the durable state: briefs, receipts, objects, and the rows that record routing outcomes.

The flow runs in a star. The relay turns English into scoped packets. The coordinator breaks work down, picks specialists, and records the decision. Specialists publish results as new file versions and notify the coordinator. A specialist does not talk to another specialist unless the coordinator opens a task-scoped edge and records it as an edge grant.

The trust boundary is strict. Agent output is untrusted input until four checks pass: schema validation, authorization, reference resolution, and version checks. A packet that fails any check produces a typed error. Nothing is guessed, and no update is partly applied.

English lives at exactly three places: the human boundary in both directions, dispute records, and approval records. Everywhere else English is the recovery ladder when packets fail, never a hidden second protocol.

The blackboard is a file tree, and that is a convention, not a service. The shared state is a directory of briefs, receipts, objects, and routing rows. Nothing in this release runs as a background service, and no live multi-agent deployment was used to verify the release. Verification was static and single-machine. Section 15 says exactly what that leaves open.

4. Rules that keep compression safe

Eight rules keep short messages honest.

1. **State change only.** Name the belief, decision, plan, or action the receiver should change. If none exists, stay silent.

2. **One primary act.** Each packet carries one communicative purpose. Independent acts become independent packets.
3. **References over repetition.** Store evidence and artifacts once. Send ids, versions, hashes, and the requested action.
4. **Deltas over snapshots.** Mutate known state against an explicit base version. Ask for a snapshot when the base is missing.
5. **Claims are not evidence.** Consequential assertions link to supporting and opposing evidence. Provenance stays in the objects.
6. **Dissent survives synthesis.** Record competing positions separately. Resolution appends a decision; it never deletes the losing view.
7. **Fail closed.** Unknown codes, stale state, unresolved references, and invalid approval never trigger guessed behavior.
8. **English is a recovery channel.** Plain language is for protocol failure, safety-critical ambiguity, audit explanation, and human review.

Protected meaning outranks compression

Some fields must never be dropped when they can change an outcome. The specification groups them as P0 to P9.

Class	Protected content	Carried by
P0-P2	Safety constraints, critical evidence, counter-evidence	cond, ev, ce, and object types k, e, z
P3-P5	Dissent, scope, provenance	dispute objects with all positions preserved, st:"disputed", deps and sup, scope refs in s
P6-P9	Tool uncertainty, irreversibility, approval, version and integrity	err, cov, cf, auth, base, hash, exp

An encoder may shorten or reference these fields. It must never remove them. Dropping a protected field to save tokens is a defect, not an optimization.

The test that keeps the whole thing honest: after decoding a packet, a compatible receiver must recover the same conclusion, evidence set, conditions, uncertainty, dissent, and requested action. If two receivers can read the same bytes and disagree about any of those, the encoding is wrong, however short it is.

5. The packet

The wire form is UTF-8 JSON with no insignificant whitespace. One object per line. Surrounding prose in a file is English. A packet line parses as JSON on its own, and no prefix may precede the opening brace on that line.

Seven fields are required in every packet: the wire version `v`, a sender-unique message id, a task id `tk`, a project id `prj`, a sender `f`, a single recipient `t`, and one act `a`. Every other field is conditional. It appears only when the act needs it, and absent fields are omitted, never sent as `null`.

Two names must not be confused. `SYM-2P/1.0` is the profile name. The wire field `v` is the integer 2. A major version other than 2 is rejected. The profile revision travels in `ext.sym2p.rev` and defaults to `1.0`

when absent. That entry is the only sanctioned use of `ext` in 1.0; everything else under `ext` is experimental, and a receiver that ignores it still holds a complete and safe packet.

Key order is fixed and exact

v, id, tk, prj, f, t, a, s, st, cf, err, cov, ev, ce, cond, ops, rq, base, exp, auth, ext

Twenty-one keys, in that order. Reordering is a rejection. Extra keys are a rejection. Unknown keys are a rejection: the envelope allows no additional properties. Optional fields are omitted rather than nulled.

A minimal brief packet, shown here in the pretty-printed form the repository template uses. On the wire it is the same object on one line with no insignificant whitespace.

```
{
  "v": 2,
  "id": "b441",
  "tk": "mr1",
  "prj": "sym2integration",
  "f": "cos",
  "t": "builder",
  "a": "assign",
  "s": "mr1",
  "st": "proposed",
  "cf": 88,
  "cov": 70,
  "cond": [
    "scope: implement SYM-2P/1.0 sections 3-5 only",
    "constraint: no existing path may be modified",
    "acceptance: validator exits 0 on valid fixtures, non-zero on invalid"
  ],
  "rq": "ack",
  "base": "mr1:v1",
  "exp": 40,
  "ext": {"sym2p": {"rev": "1.0"}}
}
```

The validator enforces all of this. It rejects duplicate keys, bad id grammar, float scores where integers are required, unknown enums, reordered keys, extra keys, and a wrong major version. Each rejection is a typed diagnostic naming the file, the line, the code, the field, and the message. The validator never guesses, and it never partly applies an update.

6. Vocabulary

The vocabulary is small on purpose. Boring words beat clever ones.

Enum	Values
Acts a, 12	assign, assert, question, answer, propose, accept, reject, challenge, verify, update, escalate, done
Requests rq, 9	ack, upd, ver, rev, exec, src, exp, full, decide
Statuses st, 7	observed, inferred, assumed, proposed, disputed, rejected, verified
Errors, 8	ERR:SYN, ERR:SEM, ERR:REF, ERR:VER, ERR:EXP, ERR:AUTH, ERR:AMB, ERR:POL

Enum	Values
Object type prefixes, 13	t, q, c, h, e, p, act, r, d, k, u, z, f, dis

Each packet holds exactly one act. A group send is N packets plus one routing row naming the set; multi-recipient packets are not legal in 1.0. A request states what the sender wants next, and the receiver answers with the matching behavior or a typed error.

Error codes travel in `ext.err` on error packets. The sender's `err` score stays a separate integer field, and the two are never the same thing.

Ids and refs

One grammar, verbatim:

```
ID  := [a-z][a-z0-9_-]{0,31}
REF := ID [":v" VERSION]
VERSION := [1-9][0-9]*
```

An id starts with a lowercase letter, then up to 31 lowercase letters, digits, underscores, or hyphens. A ref is an id with an optional version suffix such as `e3:v3`. A version never hides inside the identifier. No aliases, no prose inside ids, no dotted ids.

Scores are routing signals, not measurements

`cf`, `cov`, and `err` are integers from 0 to 99. `cf` marks how strongly the sender endorses a claim, `cov` how much of the relevant evidence space was examined, `err` the sender's own estimate of the chance of material error. They are routing signals only. They are not calibrated probability, and the repository suspends cross-profile score comparison: a raw score gap must not open a dispute across heterogeneous providers or models. Disputes open on evidence conflict or explicit disagreement instead. Calibration per role and task family is an open item.

Implemented subset, reserved values

The validator accepts and rejects against the full enums, because an unknown enum value is a rejection. The exercised operational subset is narrower: `assign`, `assert`, `answer`, `verify`, `update`, `done` plus `escalate` for `acts`, `ack`, `upd`, `ver`, `rev`, `exec`, `src`, `full`, `decide` for requests, and all seven statuses. The unexercised values stay reserved and legal. Removing one would be a semantic change and would need a version increment.

7. Durable state and versions

Messages notify. Files remember. Packets carry the minimum change; objects carry the content.

The project tree holds briefs, receipts, objects, and the rows that record routing outcomes. The relay owns the project slug; the coordinator owns placement.

Every object carries a header block

Metadata	Convention
Identity	id plus type plus tk plus prj
Version	monotonic integer v, plus a sup supersedes chain
Authorship	by (creating role id) and at (ISO-8601 UTC creation time or step)
Integrity	hash = sha256: <hex> over canonical content
Lineage	deps (dependency refs) and sup (derivation refs)
Governance	acl (role ids) and ttl in ephemeral, task, project, regulated
Body	the content itself, stored once; packets reference it

Published objects are immutable

A revision is a new file, never an edit in place. <id>.md is version 1. A revision is written as <id>.v<N>.md with an incremented v and a sup pointer to the prior version. Hash, lineage, and publication happen together; only then does the writer emit a short update or assert packet naming the new ref. Old versions stay available for replay and audit until retention permits deletion. Files that already exist as version 1 are grandfathered as version 1: no retrofit, no rewrite.

Core object types carry required content. Evidence holds source, scope, date, extract, and caveats. A claim holds statement, status, evidence, and counter-evidence. A plan holds steps, owners, dependencies, and stop conditions. A dispute holds positions, class, and needed resolution. A decision holds options, selected option, authority, and rationale refs. An artifact holds location, hash, producer, and validation status.

Retention and routing rows

Retention classes attach to every object. ephemeral is deleted at task close. task is kept through the audit window. project is kept across related tasks. regulated covers approvals and irreversible actions with policy-defined retention. The duration windows are an open item (O-4); the interim rule is strict. ephemeral deletes at close, task and project are never deleted without a coordinator decision, and regulated is never deleted.

Routing rows make sparse routing observable. Every dispatch records the sender and message id, the task and project, the recipient, the gate outcome as sent or silent with a reason, the assigned budget, any edge grant, and the expiry. Silence reasons include "the receiver state will not change", "duplicate or low novelty", and "expected value below cost".

One honest gap. Without the recorded outcome, sparse routing cannot be audited. The row writer is an obligation on the coordinator layer, and it is not a shipped tool. That gap is open, and Section 15 counts it.

8. Deltas, retries, and failure

A state change is an ordered set of typed operations against one known base version. The receiver applies it atomically, or not at all.

Four operations exist. `add` creates a field or appends a list item. `remove` deletes the exact expected value. `set` replaces a `from` value with a `to` value. `move` transfers ownership or position. Paths use a restricted JSON Pointer subset, and wildcard mutation is rejected.

Apply follows a locked order: authenticate the sender; dedupe on the pair of sender and message id; resolve and hash the base; validate every operation and precondition; apply all operations in memory; validate the resulting object; publish one new version; then acknowledge.

A mismatch returns `ERR:VER` with the receiver's current version and a request for full state. The sender then supplies a snapshot or recomputes the delta against the current version. Neither side merges silently, and the validator contains no merge path, so none can be invoked by accident.

Retries are idempotent. A retry keeps the original id and payload hash. A duplicate returns the prior terminal response without re-running work. The dedup key is the pair `(f, id)`.

Out-of-order packets queue only when the missing predecessor is known and bounded. Otherwise the receiver requests full state. Conflicts become disputes or a coordinator replan, and last-write-wins never applies to plans, decisions, or approvals.

Two behaviors of the shipped validator that deserve a plain statement

Knowledge is caller-supplied. Without receiver state, base fields are only syntax-checked: a run with no receiver knowledge cannot know what is current. With `--state` supplied, a stale base yields `ERR:VER` and names the current version. Reference-aware and expiry-aware checks follow the same pattern: they need caller-supplied state and a reference clock (`--require-refs`, `--now`). Without those, refs and expiry fields are syntax-checked only, and the usage skill says so in the same words.

One code is validator-local. Duplicate ids inside a single validator run yield `ERR:DUP`. That code is local to the static tool, not a wire code. The live protocol returns the prior response; a static validator has no prior response to return, so it fails closed instead. A deliberate replay passes `--allow-duplicates`, which turns the duplicate into a warning and keeps exit code 0.

9. Routing and the stop rule

The coordinator routes only information with expected behavioral value. The gate order is locked: reject unless schema, authorization, refs, version, and policy all pass. Then stay silent if the receiver state will not change; stay silent if the message is duplicate or of low novelty; stay silent if the expected value is below invocation and processing cost; send a reference when the shared object already carries the content; otherwise send the canonical packet.

The default topology is a star centered on the coordinator. Specialists publish to the tree and notify the coordinator. A specialist-to-specialist edge is temporary, scoped to one task, opened only when the receiving specialist owns the affected state, and recorded as an edge grant in the routing row. All-to-all specialist conversation stays out of version 1.

The lifecycle has six steps.

1. **Intake.** The relay creates project and task ids, scope, constraints, deliverable, and acceptance criteria.
2. **Plan.** The coordinator builds a dependency graph with owners, budgets, and stop conditions.
3. **Execute.** Specialists publish results and evidence as objects.
4. **Verify.** Required checks run independently, and disputes stay visible.

5. **Decide.** The coordinator records a decision or escalates to the relay.
6. **Close.** The relay reports in English, and the coordinator freezes audit state and applies retention.

The stop rule is explicit and recorded. Collaboration ends when acceptance criteria are met, when the marginal value of another call falls below its cost, when a hard budget is reached, or when policy requires human resolution. Invoking the rule is a recorded row, not a silent stop.

Phase 3 is held, not implemented. Novelty detection, expected-value gating, dynamic specialist selection, temporary direct edges beyond the recorded grant, and thresholds tuned per task family are named and not implemented. The predicates that decide whether a message changes receiver state, whether it is novel, and whether its value exceeds its cost are undefined and unverified. Only mechanical predicates are enforced in 1.0, and no routing-optimization code ships. This is deliberate, and the hold is carried as open item O-3.

The reason for stating the hold this plainly is that a hold is easy to sell as a plan. It is not a plan here. Until the three predicates are defined and measured, the honest status is that the routing layer is mechanical, and the sparse-routing rows that would make optimization auditable are an obligation on the using lane rather than a shipped tool.

10. Uncertainty and dissent

Confidence routes work. It does not certify truth.

Three scores travel with claims: `cf`, `cov`, and `err`, all integers 0 to 99, all routing signals until calibration exists for a specific role and task family. No score is presented as calibrated probability, and no raw score is compared across heterogeneous providers or models.

Disagreement is a durable object rather than a mood in a chat log. A dispute records its subject, a class such as fact, source, scope, evaluation, causal, value, plan, or risk, a list of positions with authors and evidence, the resolution needed, and a status. Each position keeps its author, stance, confidence, and evidence refs.

The resolution contract is strict. A resolution names the selected position or synthesized conclusion, the evidence that changed the result, the adjudicator, the remaining uncertainty, and the action impact. It preserves all prior positions. A resolved dispute whose preserved positions do not match its original positions is rejected, and the fixture suite proves that rejection with a real case.

Claims without evidence links fail the spirit of the protocol even when they pass the schema. Integrity is necessary but not sufficient: a valid packet may still carry a wrong claim, and independent evidence checks remain separate controls.

Nine open items

These are deliberate boundaries, not build failures. Each has an interim rule and an owner class. None is closed.

#	Open item	Interim rule
O-1	Transport signing. Cross-profile file handoffs are distributed transport.	Hash-only integrity, single-machine trust assumption. Revisit if an external partner joins.
O-2	Home of the authorization verifier and the approval store.	Approval objects are regulated-class objects in the project tree, and verification is a pre-execution gate performed by the executing stage and logged in the routing row.
O-3	Routing predicate definitions.	Only mechanical predicates are enforced in 1.0. Value gating is Phase 3.
O-4	Retention windows.	ephemeral deletes at task close, task and project are not deleted without a coordinator decision, regulated is never deleted.
O-5	Per-role calibration of cf, cov, and err.	Evidence-anchored dispute triggers only, on evidence conflict or explicit challenge.
O-6	Where dispute and decision objects live, and who may close a dispute.	The stage owner may close a dispute inside their domain; the coordinator may close it otherwise and must record the adjudicator and the remaining uncertainty.
O-7	Minor-version convention beyond ext.sym2p.rev.	v stays 2; profile revisions that add no required field are legal, and a receiver may accept them only while all unknown extensions are optional.
O-8	Fallback logging surface.	The fallback event is a routing row, and a QA verdict must report fallback and parse rates.
O-9	Group addressing.	Locked for 1.0 as single-recipient: a group send is N packets plus one routing row naming the set. Reopens only with a version increment.

11. The human boundary and authorization

The relay does more than paraphrase. It preserves task identity, project routing, acceptance criteria, uncertainty, alternatives, unresolved disagreement, and authorization status across the English boundary.

Inbound, from human to team: identify the project and its coordinator; classify the request as a new task, update, decision, correction, cancellation, or question; extract scope, constraints, deliverable, deadline, budget, and acceptance criteria; preserve ambiguity as an explicit unknown rather than filling it silently; and emit one or more scoped packets.

Outbound, from team to human: read the authoritative objects and the coordinator's decision; state the result first in plain English; surface material uncertainty, options, dissent, blockers, and required decisions; never expose internal packets or private reasoning; and keep traceability through internal references.

The authorization envelope

An accept packet names the action ref, requests exec, and carries an auth object:

```
{
  "v": 2, "id": "m102", "tk": "mr1", "prj": "sym2integration", "f": "relay", "t": "cos",
  "a": "accept", "s": "act88:v1", "rq": "exec",
  "auth": {
    "kind": "human_approval", "scope": "act88:v1",
    "approved_at": "2026-09-17T02:00:00Z", "expires_at": "2026-09-17T03:00:00Z",
    "ref": "approval:a881"
  }
}
```

The approval record stores the exact approved action and scope in English. The packet carries the reference, never the private conversation text. The executor verifies that subject, version, scope, and expiry all match.

Approval is required for external sends, publications, purchases, bookings, account changes, deletions, and other irreversible or consequential operations. It is required for any step that expands audience, destination, data scope, access, cost, or recurrence. It is required for any policy-defined high-risk operation even when a specialist or the coordinator recommends it. There is no authority by implication: a prior approval never authorizes a broader action, a different destination, a later recurring job, or a changed object version. Verification of feasibility is not authorization to act.

Two limits, stated plainly. No full platform authorization store ships here; the requirement itself is locked at 100 percent of irreversible actions carrying matching, unexpired authorization. The home of the verifier and the approval store is an open item (O-2). Expiry checks also need a caller-supplied clock: without --now, the validator checks the shape of the envelope and nothing more.

12. Errors and the fallback ladder

Every failure has a typed response and a safe downgrade.

Code	What it means	What happens
ERR:SYN	Bad JSON or schema	Rejected, with the field named and the minimum recovery requested
ERR:SEM	Contradictory meaning	Rejected, for example a challenge without counter-evidence
ERR:REF	Unresolved reference	The missing object is requested
ERR:VER	Base or version mismatch	The current version is returned and full state is requested
ERR:EXP	Expired message or approval	Never executes
ERR:AUTH	Identity or authority failure	Denied by default, including cross-project reads
ERR:AMB	Unsafe ambiguity	Escalates to the coordinator or the human
ERR:POL	Policy restriction	Rejected, for example a multi-recipient packet

Error packets use the standard envelope. An answer packet names the subject, requests full state, states the attempted base, and carries the code plus the receiver's current version in ext:

```
{
  "v": 2, "id": "m105", "tk": "mr1", "prj": "sym2integration", "f": "cos", "t": "builder",
  "a": "answer", "s": "f88", "rq": "full", "base": "f88:v1",
  "ext": {
    "err": "ERR:VER", "have": "f88:v3", "sym2p": { "rev": "1.0" }
  }
}
```

The recipient names what failed and the recovery it needs. It never guesses the intended state, and it never applies a partial operation.

The fallback ladder

Four rungs: canonical SYM-2P, then verbose JSON under the same schema, then concise English with explicit meaning, then coordinator or human adjudication. Each downgrade is logged with the original packet, the failure code, the recovered interpretation, and the outcome. A fallback never weakens authorization or protected meaning.

Security controls, named as controls

Stable agent identities. Project- and task-scoped access. Canonical hashes. Expiry and replay protection. Size limits. Immutable logs. Schema allowlists with unknown keys isolated under `ext`. External content treated as data, never as instructions. Tool actions checked against task and authority. Sensitive fields referenced, minimized, and access-controlled. Cross-project reads denied by default. Fail closed when a policy service fails.

The control that is missing is the one that would make the rest of this list a platform. Transport signing is not implemented (O-1). Integrity rests on a sha256 computed in process, under a single-machine trust assumption. That is enough for one machine writing files to itself, and it is not enough for a cross-party handoff.

13. Prompt contracts

Agents that speak this protocol share a stable prefix. The static specification, the enums, the validation rules, and the protected meaning come first; dynamic task context comes after. The suggested order is shared protocol prefix, role block, project policy, current task envelope, referenced state, and the latest message.

Static content stays byte-identical where practical, so platforms that reuse prefixes can benefit. No correctness or cost claim depends on that reuse. Prompt-prefix reuse is an optimization, never a channel.

The shared prefix tells the agent the essentials. Communicate in compact JSON only. Emit one object per message. Use canonical key order. Never invent enums, ids, versions, or evidence. Reference objects instead of repeating bodies. Send only when a receiver state should change. Preserve safety, evidence, counter-evidence, scope, provenance, dissent, authorization, version, and integrity fields. Fail closed with a typed error on invalid or ambiguous input. Never expose private reasoning.

Role blocks specialize. The coordinator decomposes tasks, assigns owners with acceptance criteria and budgets, routes only material novelty, holds authoritative routing state, opens disputes, requires verification where policy or uncertainty demands it, and synthesizes results. It never overrides a stage verdict inside that stage's domain; it opens a dispute or escalates instead. The specialist reads only assigned objects and dependencies, returns claims with status, evidence, uncertainty, and the requested next action, publishes durable output and sends a reference, and never claims completion against unmet criteria. The relay speaks English to the human and packets to coordinators, maps every request to project and task ids, preserves scope, uncertainty, and disagreement, renders state as plain

English, and issues approval references only for actions the human explicitly approved with exact scope and expiry.

No latent channel

Text JSON over durable file artifacts is the only inter-agent surface. Hidden-state capture, embedding exchange, and key-value cache exchange are out of version 1, and no supported hidden-state or cache channel is assumed. Cross-profile execution is profile-native, so nothing may be assumed shared beyond what is written to disk: no shared memory, no shared process state, no cross-language context. If it is not in an object, it does not exist.

Canonical key order buys comparability, signing, and logging stability. It is a validation discipline, not a fleet-wide guarantee about how a model generates text.

14. Worked flow

A review, fix, verify, and merge loop shows the protocol in motion. The shipped file `examples/protean-sym2p/review-fix-verify-merge.jsonl` holds twelve lines, each one canonical packet or object.

- The relay assigns the task. Packet `m101` carries scope, acceptance, and constraints in `cond`: review the pull request, fix the failing merge gate, re-verify, then merge. No force-push, and no history rewrite without new approval.
- The relay attaches approval. Packet `m102` accepts `act88:v1` and requests `exec`, with an auth envelope scoped to that version and a one-hour window. The merge is authorized for one version and one window; feasibility work elsewhere is not authorization.
- The coordinator assigns the fix. Packet `m103` sends a builder the subject with `st: "proposed"`, confidence and coverage signals, conditions, and an ack request. The control plane stays untouched by explicit condition.
- A stale delta fails closed. Packet `m104` attempts an update against `f88:v1`. The coordinator answers in `m105` with `ERR: VER`, names have: `"f88:v3"`, and requests full state. No state is applied, and the sender recomputes against the current version. No silent merge occurs.
- The corrected delta lands and is verified. Packet `m106` updates against `f88:v3` with typed operations. Packet `m107` verifies the new version with evidence refs `e31` and `e32` and `st: "verified"`.
- A dispute is preserved. Packet `m108` challenges with counter-evidence `e33`. Object `dis45` keeps both positions, names its class and needed evidence, and carries a resolution that selects one position, cites the changed evidence, names the adjudicator, states the remaining uncertainty, and preserves both original positions.
- The merge executes and the loop closes. Packet `m109` executes the update with matching auth for the same scope and window. Packet `m110` closes with `done`, evidence refs, and an update request. Object `dec45` records the decision with options, authority, rationale refs, and an English summary for the human, including the residual risk and the follow-up ticket.

Validate the file directly, and in strict canonical mode; both runs exit 0.

This example is a convention demonstration, not a deployment log. It proves the shapes compose. It does not prove fleet behavior.

15. Implementation status

Three phases were named in the original plan. Their status is the spine of this report.

Phase	What it covers	Status
1 - Instrument and constrain	Ids, purpose labels as acts, authorization events, the state-change-only rule, with English coordination as the baseline	Already the operative mode. Locked where it intersects the packet and object sections
2 - Blackboard and protocol	Schema validation, object storage, versioning, references, deltas, idempotency, error packets, the fallback ladder	Adopted and implemented. This is the implemented scope of the release
3 - Routing optimization	Novelty detection, expected-value gating, dynamic specialist selection, temporary direct edges, prompt-prefix reuse, thresholds per task family	Held, not implemented. Named, out of scope, predicates undefined and unverified

15.1 Implemented behavior, enforced in code

- The validator enforces the exact key order, the seven required fields, the full enums, integer score ranges, and the id and ref grammar. It rejects duplicate keys, floats where integers are required, unknown enums, extra keys, reordered keys, and a wrong major version.
- The canonicalizer emits compact UTF-8 with absent optionals omitted, never null.
- With receiver state supplied, a stale base returns `ERR:VER` with the receiver's current version; `--require-refs` enforces resolvable refs; `--now` enforces authorization expiry. Every rejection is a typed diagnostic naming file, line, code, field, and message.
- `--strict-canonical` proves canonical serialization; a non-canonical stream is rejected against it and the fixture suite covers that case.
- Duplicates in one run return `validator-local ERR:DUP`; `--allow-duplicates` downgrades the duplicate to a warning and keeps exit code 0.
- All eight wire codes are typed, and exit codes are stable: 0 every document valid, 1 at least one invalid, 2 usage or IO error.
- Both packet templates validate, and the worked example validates in auto and strict canonical modes.
- The fixture suite gives every valid fixture a pass and every malformed, stale, duplicate, or protected-semantic fixture a typed rejection: 4 valid fixtures, 19 invalid fixtures, and a receiver-state fixture.
- The installer supports a dry run, prints every path it would write, writes nothing in that mode, and refuses to run without a target. It reports zero network calls and uses bash and coreutils only.
- The leak gate scans text surfaces for private identifiers by digest and for machine-specific absolute path shapes.
- Evidence files under `AUDIT/protean-sym2p/` ship with the repository: a provenance note, an audit README, and a verification log.

15.2 Conventions and recommendations, specified but not mechanically enforced

Object header blocks. Immutable revision files and supersedes chains. Grandfathering of existing version 1 files. Retention handling. The shape of routing rows. Recording the stop rule. Fallback logging as rows. Approval objects as regulated files. These are specified in the specification and must be followed by hand or by lane tooling. The repository does not enforce all of them mechanically, and this report does not claim that it does.

15.3 Unverified platform assumptions, not simulated

Stable identities, a namespace service, a persistent store beyond files, an authorization hook, platform-scoped access control, a hash service beyond in-process sha256, replay protection beyond dedupe keys, and immutable logs beyond file discipline have no platform provider in this release.

Version 1.0 substitutes file conventions for storage and namespace, a sha256 computed in process for hashes, and file immutability for logs. Transport signing is absent. Confidence is uncalibrated. No claim in this report depends on a live service, a live deployment, or a calibrated score.

15.4 Receipts

These commands were run from the repository root on the revision this report was built from, on macOS with Python 3.9.6. The audit verification log inside the repository records the same commands as they behaved at projection time; where a count differs, both are honest measurements of different trees.

Command	Observed result
<code>python3 tests/run-tests.py</code>	exit 0, 30/30 cases passed, ALL PASS
<code>python3 -m unittest discover -s tests</code>	exit 0, Ran 7 tests, OK
<code>python3 gates/protean-sym2p/check-internal-names.py .</code>	exit 0, clean: 47 files scanned, 0 findings
<code>python3 gates/check-internal-names.py .</code>	exit 0, clean: 49 files scanned, 0 findings
<code>python3 scripts/protean-sym2p/sym-validate.py --help</code>	exit 0, usage surface with all options listed
<code>python3 scripts/protean-sym2p/sym-validate.py --quiet <templates and example></code>	exit 0, no output
<code>python3 scripts/protean-sym2p/sym-validate.py --strict-canonical --quiet <example></code>	exit 0, no output
<code>python3 scripts/protean-sym2p/sym-validate.py --state <state.json> <stale-base.jsonl></code>	exit 1, ERR:VER: base: stale base f88:v1; receiver holds f88:v3 with the resend instruction
<code>python3 scripts/protean-sym2p/sym-validate.py <absent file></code>	exit 2, ERR:SYN: cannot read file
<code>bash install.sh --target <dir> --dry-run</code>	exit 0, network calls: 0, seven install targets, result: dry-run, no files written

The two gate variants are both shipped: the repository-local gate and the install-portable copy under `gates/protean-sym2p/`. They differ only in how they locate the blocklist, so they report slightly different file counts on the same tree.

15.5 The audit files, and what they are not

`AUDIT/protean-sym2p/` carries evidence and provenance. Nothing there is normative. Where an audit file and `SPEC.md` disagree, `SPEC.md` governs and the disagreement is a defect in the audit directory.

The repository is a neutral projection of its private source material. Artifact headers, author lines, absolute paths, session identifiers, and internal record citations were removed. Role identifiers became neutral placeholders (`relay`, `cos`, `research`, `arch`, `design`, `builder`, `qa`), each a legal id under the locked grammar. Task identifiers were renamed to neutral values, and object hashes in the worked example were recomputed after the renames so the example stays internally consistent. The wire field, the required field set, the key order, the grammar, the enums, the error codes, the failure semantics, the protected semantics, and the acceptance criteria were carried across unchanged. No open item was resolved in the projection.

The leak gate is the enforcement surface for that neutrality. It has two detectors. The first hashes every token in every text file and compares the digests against a blocklist that holds digests only, so the gate file itself never contains a private name. The second matches machine-specific path shapes by regex written so that no literal example of a blocked path appears in the gate. Binary extensions such as `.pdf`, images, fonts, and archives are skipped, so the gate guards text surfaces rather than everything in the tree.

16. How to use the repository

Install

```
bash install.sh --target <dir>
bash install.sh --target <dir> --dry-run
```

Bash and coreutils only. Zero network calls. Every written path is printed. No target means no run. A dry run prints the plan and writes nothing. The payload is seven install targets: `SPEC.md`, `skills/sym2p/`, `scripts/protean-sym2p/`, `templates/protean-sym2p/`, `examples/protean-sym2p/`, `AUDIT/protean-sym2p/`, and `gates/protean-sym2p/`.

Used through a composer, the ingredient is fetched once from its pinned tag and reused from a content-addressed cache keyed by the commit SHA. The composer's `--offline` mode performs zero network calls and fails closed when the cache entry is absent. The installer itself never reaches the network at all.

Read the norm first

`SPEC.md` is the norm. The usage skill at `skills/sym2p/SKILL.md` documents the shipped implementation and states no rule the specification does not already state. `AUDIT/` carries evidence and provenance only.

Validate

```
python3 scripts/protean-sym2p/sym-validate.py --help
python3 scripts/protean-sym2p/sym-validate.py <packet-or-stream>
python3 scripts/protean-sym2p/sym-validate.py --state <receiver-state.json>
  <packets.jsonl>
python3 scripts/protean-sym2p/sym-validate.py --require-refs --state <state.json>
  <packets.jsonl>
python3 scripts/protean-sym2p/sym-validate.py --now 2026-09-17T03:00:00Z <packet.json>
python3 scripts/protean-sym2p/sym-validate.py --strict-canonical <packets.jsonl>
python3 scripts/protean-sym2p/sym-validate.py --canonicalize <packets.jsonl>
```

Exit 0 means every document is valid. Exit 1 means at least one is invalid, with typed diagnostics printed. Exit 2 means a usage, IO, or configuration error. Files ending in `.jsonl` are read one object per line, which is the wire carrier; any other suffix is tried as one whole object first, then line by line.

`--canonicalize` normalizes a document before you write it as a packet line, and `--strict-canonical` proves canonical form in review.

Start from the templates, study the example, run the suite

`templates/protean-sym2p/brief-packet.json` is a canonical assign brief and `templates/protean-sym2p/receipt-packet.json` is a canonical done receipt. Both are pretty-printed for humans and stay valid JSON. `examples/protean-sym2p/review-fix-verify-merge.jsonl` walks review, a state-base failure, a corrected delta, verification, a challenge, a dispute, authorized execution, done, and a decision.

```
python3 tests/run-tests.py
python3 -m unittest discover -s tests
python3 gates/protean-sym2p/check-internal-names.py .
```

The focused suite runs real validator invocations and asserts exit codes and typed diagnostics. Fixtures live under `tests/fixtures/valid`, `tests/fixtures/invalid`, and `tests/fixtures/state`. The gates are declared in `protean-ingredient.json` and run from the repository root; the continuous integration workflow runs the installer dry run, the declared gates, and the unittest suite on Python 3.9 and 3.11. The ingredient requires nothing and recommends a separate operations ingredient for the record schemas that check routing rows; that recommendation is documentary and never affects install order.

17. Limits to build within

- Transport signing is absent. Integrity is a canonical sha256 computed in process, under a single-machine trust assumption.
- Reference and expiry checks need caller-supplied state and a reference clock. Without them, those fields are syntax-checked only.
- Routing rows and the stop-rule record are obligations on the using lane, not shipped tools.
- Phase 3 routing predicates are undefined, and no routing-optimization code ships.
- Calibration data does not exist, so no score may be read as a probability and no raw score may be compared across profiles.
- Nine open items ship open: O-1 to O-9.
- Verification is static and single-machine. No live deployment was used, and no live service is claimed.

Build inside those fences and the protocol holds. Build outside them and the report is not the thing that breaks; the record is.

Appendix A. Quick reference

Canonical key order (21 keys, exact):
v, id, tk, prj, f, t, a, s, st, cf, err, cov, ev, ce, cond, ops, rq, base, exp, auth, ext

Required on every packet: v, id, tk, prj, f, t, a
Absent optionals: omitted, never null

Acts (12): assign assert question answer propose accept reject
challenge verify update escalate done

Requests (9): ack upd ver rev exec src exp full decide

Statuses (7): observed inferred assumed proposed disputed rejected verified

Errors (8): ERR:SYN ERR:SEM ERR:REF ERR:VER ERR:EXP ERR:AUTH ERR:AMB ERR:POL

Object types (13): t q c h e p a c t r d k u z f d i s

Retention: ephemeral task project regulated

Ops: add remove set move (restricted JSON Pointer subset)

Grammar: ID := [a-z][a-z0-9_-]{0,31}
REF := ID ["v" VERSION]
VERSION := [1-9][0-9]*

Scores: cf, cov, err are integers 0-99 and routing signals only,
never calibrated probability.

Wire value v is the integer 2. SYM-2P/1.0 is the profile name.
Profile revision travels in ext.sym2p.rev, default 1.0.

Exit codes: 0 valid, 1 invalid (typed diagnostics), 2 usage / IO error.
ERR:DUP is validator-local, not a wire code.

Change control. Any field, enum, or semantic change needs a profile or major version increment, a decoder migration, partner-swap and replay tests, and continued access to prior logs. Experimental fields stay namespaced under ext until promoted. A receiver rejects unknown major versions, and may accept a newer minor profile only when every unknown extension is optional and the base packet stays complete and safe.

Appendix B. What moved from the earlier drafts

This report folds two earlier documents together: an eighteen-page implementation handoff and a twelve-page integration brief. The table below is the compact map. It is here for readers who knew the earlier documents; the rest of this report stands on its own.

Earlier material	Where it lives now	What changed
Executive decision	Section 1, plus status in Sections 9 and 15	The core bet survived; the decision is now stated as a positioning triple, with an explicit scope fence: Phase 2 in, Phase 3 held
Architecture and roles	Section 3	All four roles and the star survived; the blackboard is now a file tree, and naming is neutral
Protocol invariants	Sections 4 and 10	All eight rules survived; protected classes are labeled P0 to P9 and tied to named fields

Earlier material	Where it lives now	What changed
Packet model and field dictionary	Sections 5 and 6	The wire form survived; v is now distinct from the profile name, revision travels in <code>ext.sym2p.rev</code> , and the implemented subset is named
Shared state and retention	Section 7	Store-once-and-reference survived; object header blocks, immutable revisions, supersedes chains, and grandfathering were added
Synchronization and retries	Section 8	Typed operations and atomic apply survived; the gate order and apply order are locked, and <code>ERR:DUP</code> is documented as validator-local
Orchestration and lifecycle	Section 9	The six-step lifecycle and the stop rule survived; only mechanical predicates are enforced in 1.0
Epistemics, uncertainty, dissent	Section 10	The three scores and the dispute object survived; global default thresholds are gone, and the nine open items are carried open
Human boundary and authorization	Section 11	Inbound and outbound steps and the auth envelope survived; the interim rule for the verifier home is explicit
Reliability and error handling	Section 12	The eight codes and the four-rung fallback ladder survived; transport signing is named as absent
Agent contracts	Section 13	The shared prefix and role blocks survived; prefix reuse is labeled an optimization, never a channel
Worked flow	Section 14	The pattern became a shipped twelve-line example with a stale-base failure and its typed reply
Delivery plan and evaluation	Section 15	The three phases survived as the migration boundary; implemented behavior, conventions, and unverified assumptions are now separated
Reference appendix	Appendix A	Canonical order and enums are printed once for copy use
Schema starter, go-live list	Section 16	The sketch became a strict validator with typed diagnostics; go-live is replaced by exact commands and observed outputs
Integration brief, all pages	Sections 1 to 16	The file-native mapping and the hold posture survived; neutral roles, neutral ids, recomputed hashes, the standalone installer, the leak gate, and the audit files are new in the repository

Nothing in this map closes an open item. The nine open items travel unchanged, and the Phase 3 hold travels unchanged with them.